

Parallel Processing

Thoai Nam

High Performance Computing Lab (HPC Lab)

Faculty of Computer Science and Engineering

HCMC University of Technology



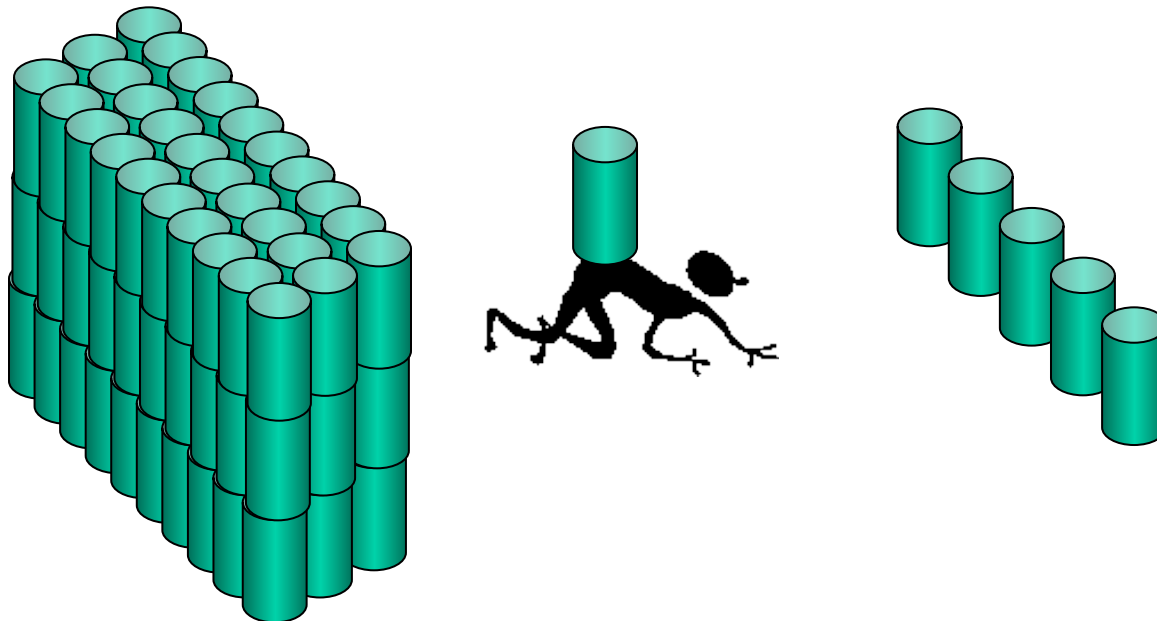
How to do

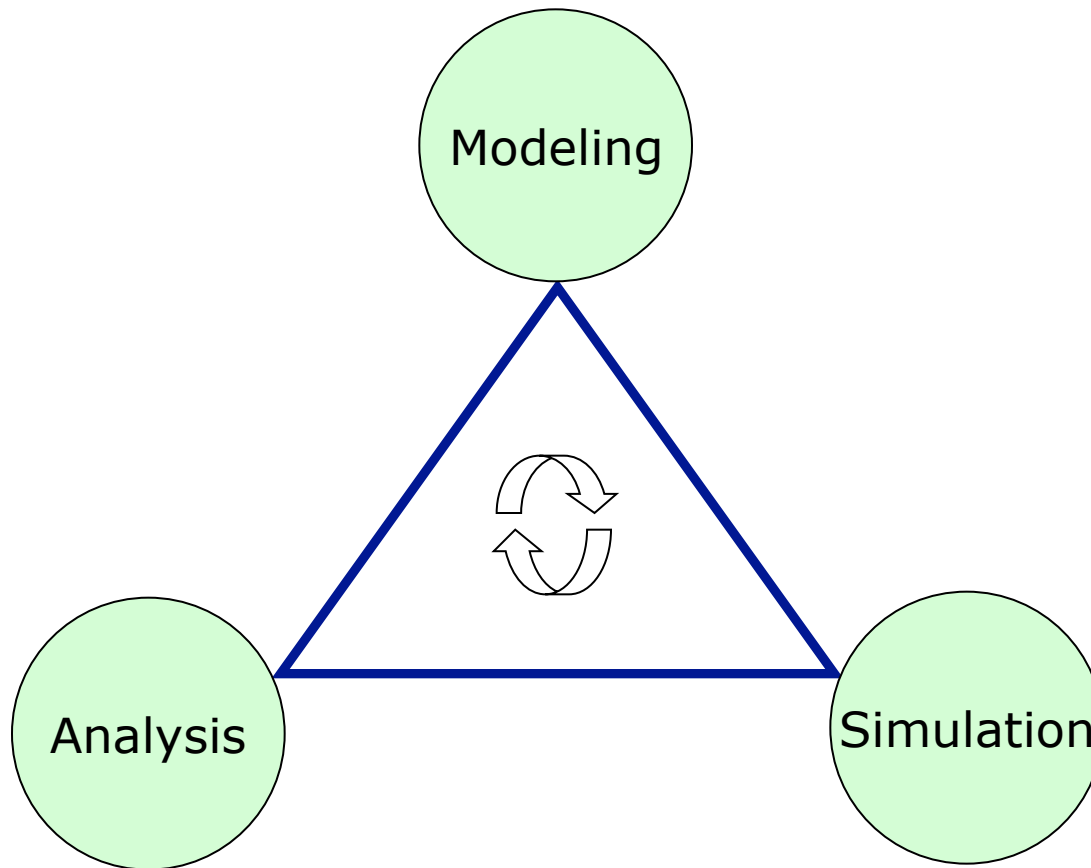
Parallel processing



Sequential Processing

- ❑ 1 CPU
- ❑ Simple
- ❑ Big problems???



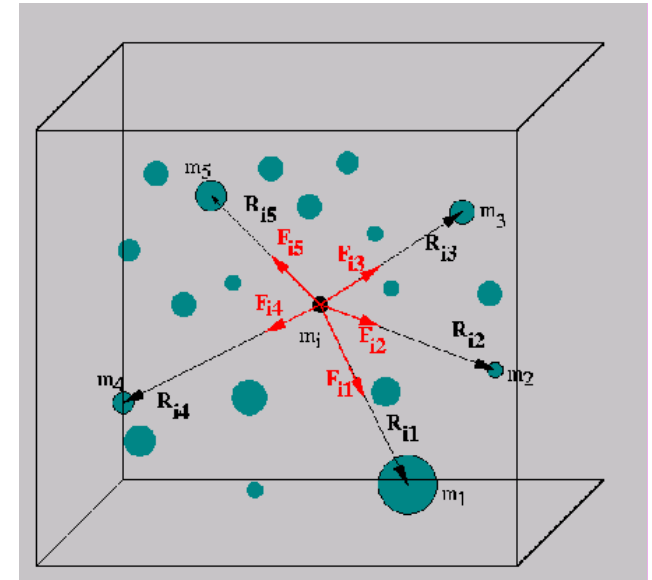




Grand Challenge Problems

- ❑ A grand challenge problem is one that cannot be solved in a reasonable amount of time with today's computers
- ❑ Ex:
 - Modeling large DNA structures
 - Global weather forecasting
 - Modeling motion of astronomical bodies

- ❑ The N^2 algorithm:
 - N bodies
 - $N-1$ forces to calculate for each bodies
 - N^2 calculations in total
 - After the new positions of the bodies are determined, the calculations must be repeated





- 10^7 stars and so 10^{14} calculations have to be repeated
- Each calculation could be done in $1\mu\text{s}$ (10^{-6}s)
- It would take ~**3 years** for one iteration (~26800 hours)
- But it only takes **10 hours** for one iteration with **2680** processors



Solutions

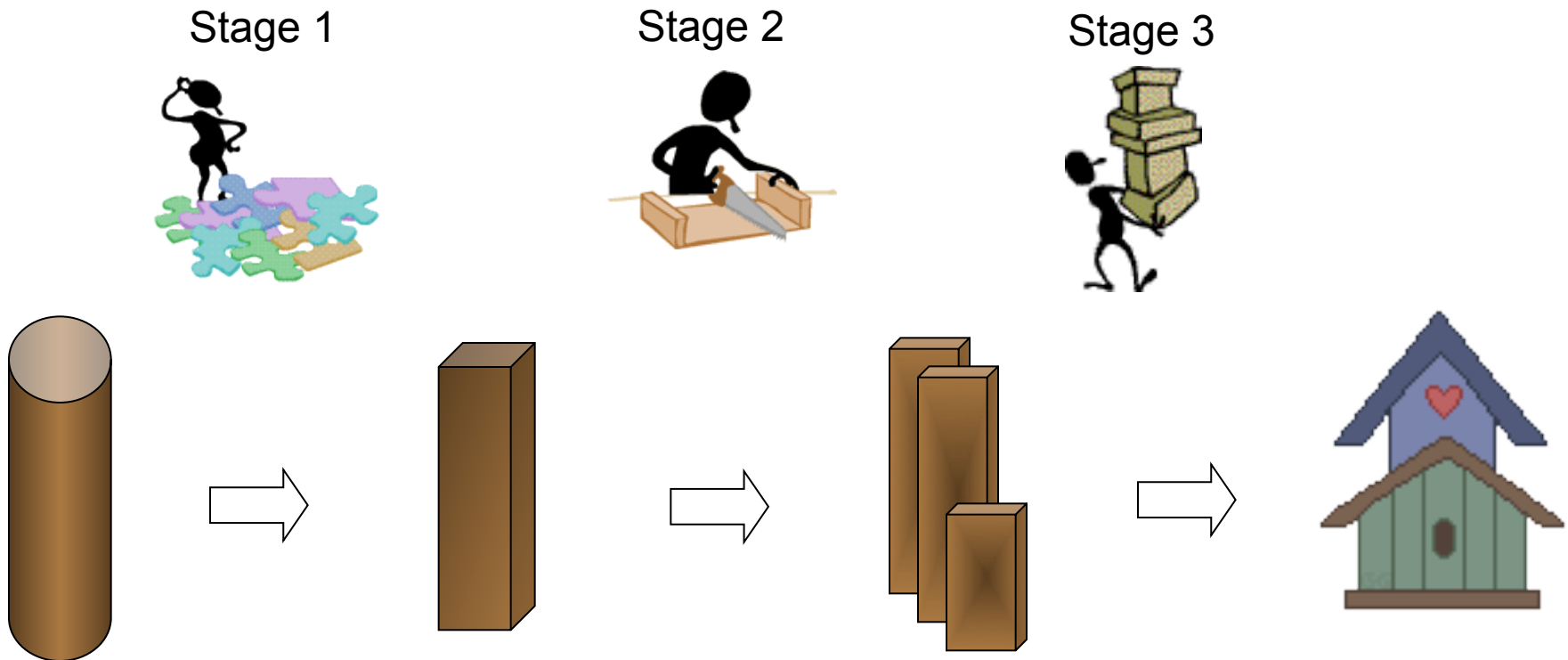
- ❑ Power processor
 - 50 Hz -> 100 Hz -> 1 GHz -> 4 Ghz -> ... -> Upper bound?
- ❑ Smart worker
 - Better algorithms
- ❑ Parallel processing



Parallel Processing Terminology

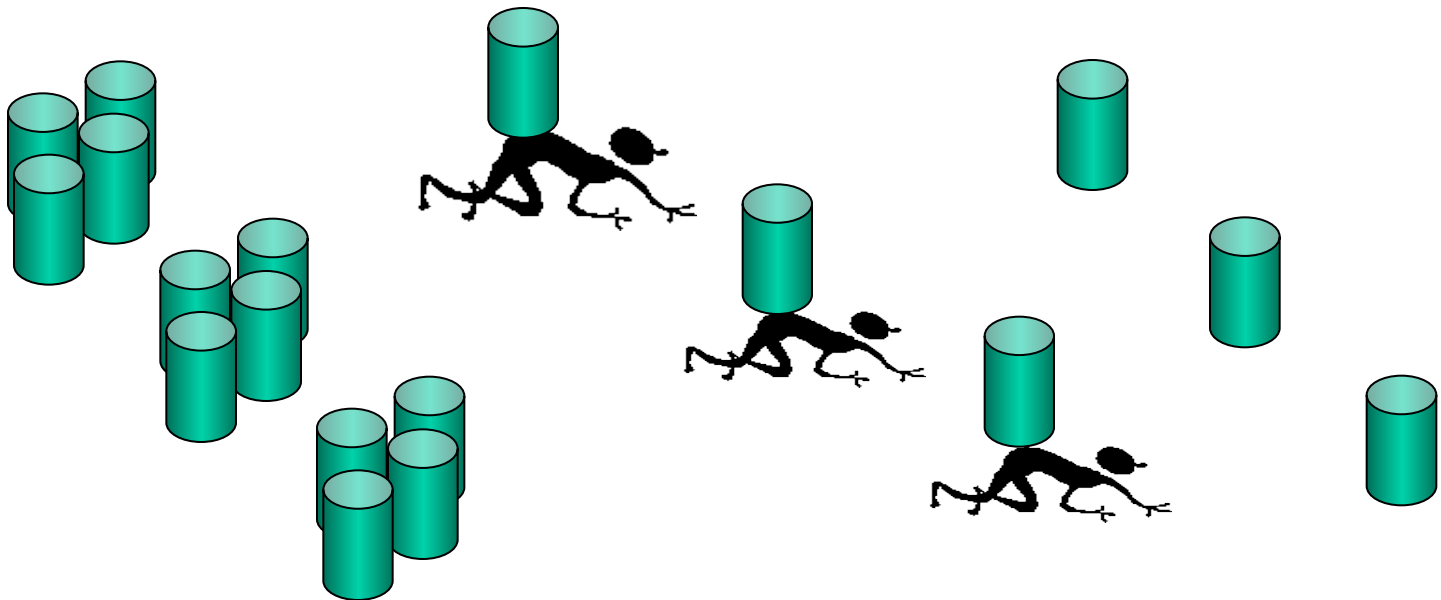
- ❑ Parallel processing
- ❑ Parallel computer
 - Multi-processor computer capable of parallel processing
- ❑ Throughput:
 - The throughput of a device is the number of results it produces per unit time.
- ❑ Speedup
 - $S = \text{Time}(\text{the most efficient sequential algorithm}) / \text{Time}(\text{parallel algorithm})$
- ❑ Parallelism:
 - Pipeline
 - Data parallelism
 - Control parallelism

- ❑ A number of steps called **segments** or **stages**
- ❑ The output of one segment is the input of other segment



- ❑ Distributing the data across different parallel computing nodes

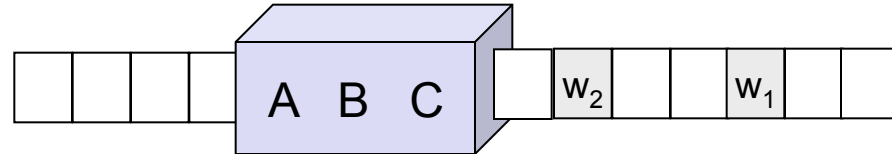
Applying the same operation simultaneously to elements of a data set



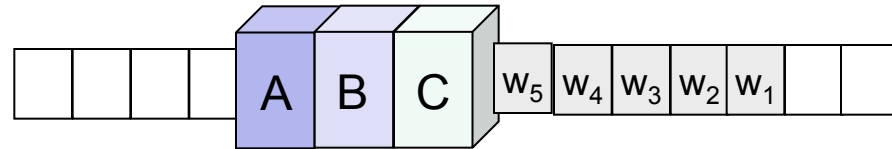


Pipeline & Data Parallelism

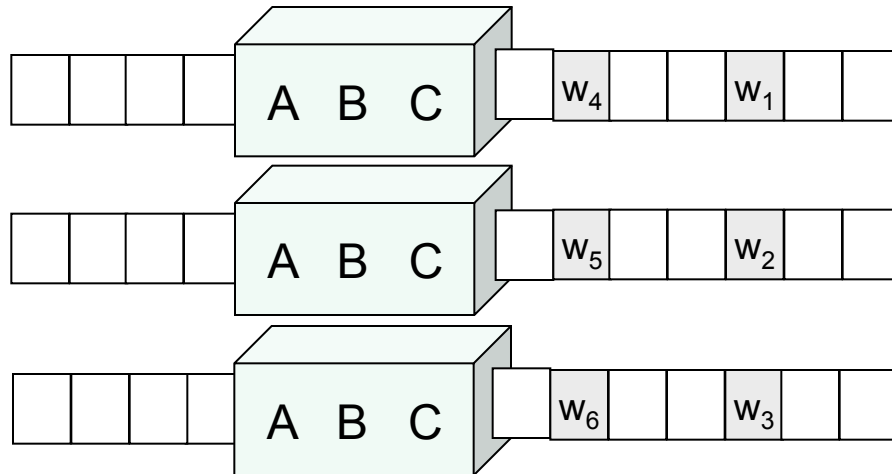
1. Sequential execution



2. Pipeline



3. Data Parallelism



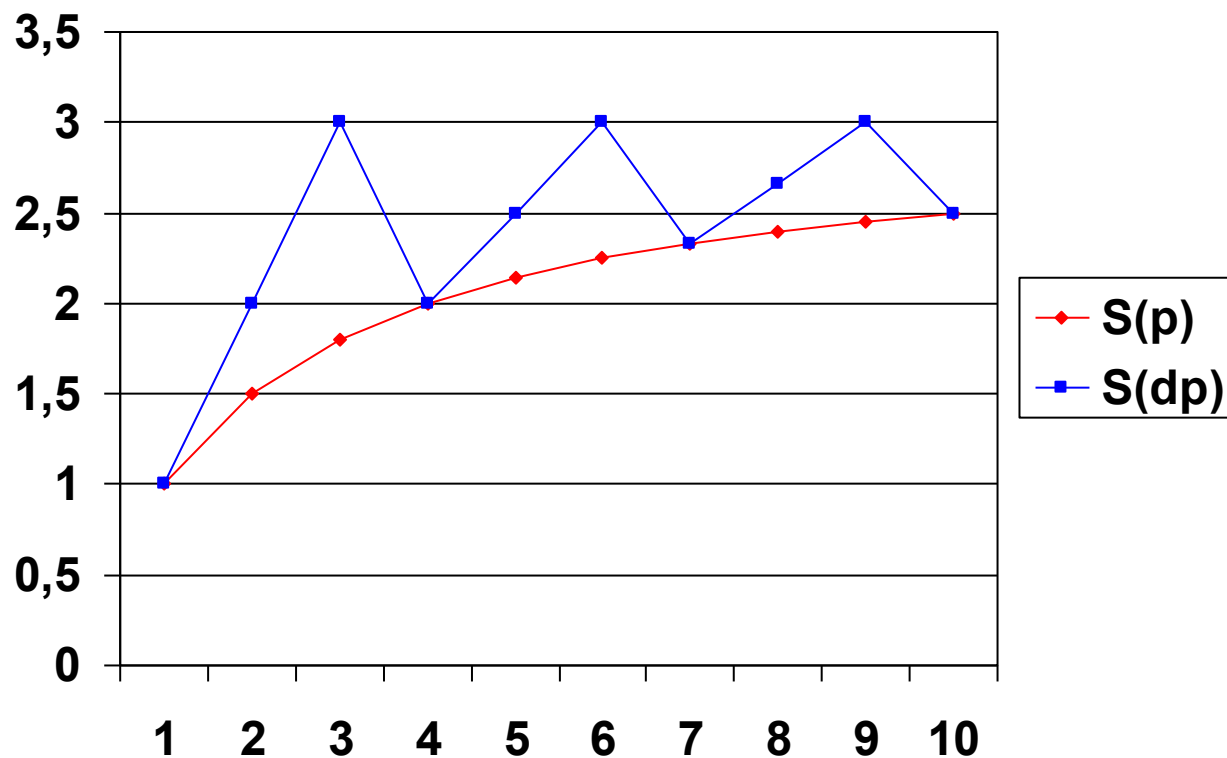


Pipeline & Data Parallelism

- ❑ Pipeline is a special case of control parallelism
- ❑ $T(s)$: Sequential execution time
 $T(p)$: Pipeline execution time (with 3 stages)
 $T(dp)$: Data-parallelism execution time (with 3 processors)
 $S(p)$: Speedup of pipeline
 $S(dp)$: Speedup of data parallelism

Widget	1	2	3	4	5	6	7	8	9	10
$T(s)$	3	6	9	12	15	18	21	24	27	30
$T(p)$	3	4	5	6	7	8	9	10	11	12
$T(dp)$	3	3	3	6	6	6	9	9	9	12
$S(p)$	1	$1+1/2$	$1+4/5$	2	$2+1/7$	$2+1/4$	$2+1/3$	$2+2/5$	$2+5/11$	$2+1/2$
$S(dp)$	1	2	3	2	$2+1/2$	3	$2+1/3$	$2+2/3$	3	$2+1/2$

Pipeline & Data Parallelism

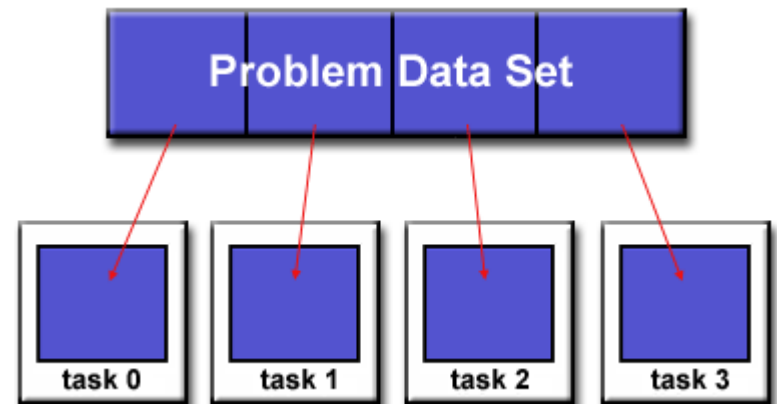




Control Parallelism

- ❑ Task/Function parallelism
- ❑ Distributing execution processes (threads) across different parallel computing nodes

Applying different operations to different data elements simultaneously





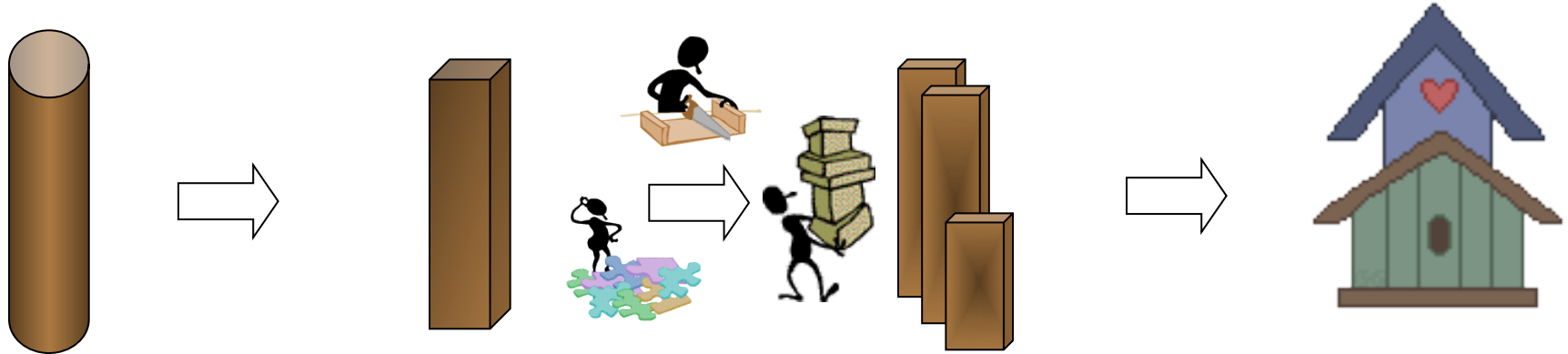
Example

{Milk, Sugar, Bread}

{Milk, Sugar, Bread, Tea}
{Bread, Milk, Coffee, Meat}
{Milk, Sugar}
{Milk, Bread, Sugar, Salt}
{Apple, Orange, Banana, Sugar, Milk}
...
{Milk, Bread, Sugar, Beer}

- Pipeline?
- Control parallelism?
- Data parallelism?

Throughput: Woodhouse problem



- ❑ 5 persons complete 1 woodhouse in 3 days
- ❑ 10 persons complete 1 woodhouse in 2 days
- ❑ How to build 2 houses with 10 persons?
 - (1) 10 persons building the 1st woodhouse and then the 2nd one later (sequentially)
 - (2) 10 persons building 2 woodhouses concurrently; it means that each group of 5 persons complete a woodhouse



Throughput

- ❑ The **throughput** of a device is the number of results it produces per unit time

- ❑ **High Performance Computing (HPC)**
 - Needing large amounts of computing power for short periods of time in order to completing the task as soon as possible

- ❑ **High Throughput Computing (HTC)**
 - How many jobs can be completed over a long period of time instead of how fast an individual job can complete



Scalability

- ❑ An algorithm is scalable if the level of parallelism increases at least linearly with the problem size.
- ❑ An architecture is scalable if it continues to yield the same performance per processor, albeit used in large problem size, as the number of processors increases.
- ❑ Data-parallelism algorithms are more scalable than control-parallelism algorithms